COLLEGE ENGINEERING MINI-PROJECT

EventPulse

Advanced Databases (AD) Academic Mini-Project Handbook

A Complete Architecture, Database Engineering & Viva Preparation Guide for the Full-Stack MERN Event Booking Platform (Derived & Adapted from Eventora-MERN)

MONGODB AGGREGATIONS · ATOMIC CONCURRENCY · PARTIAL UNIQUE INDEXES · QUERY EXPLAIN EXECUTION

Subject:	Advanced Databases (AD) / CS
Architecture:	MERN Stack (MongoDB, Express, React, Node.js)
Reference Repo:	`ShivaMani02/Eventora-MERN`
Enhanced Repo:	`jayraj0975/eventpulse-mern`
Primary Objective:	Verifiable Database Internals, Concurrency & Analytics
Document Audience:	Student Presenters, Viva Candidates & Faculty Evaluators
Academic Session:	Academic Year 2026

Public-Safe Educational Document — Zero Real Secrets, Credentials or Production Keys Included.
Contains Complete System Flowcharts, Authentic Visuals, Mongoose Schemas, Pipeline Math & 35 Viva Q&As.

Table of Contents

Chapter 1: Project Overview & Academic Scope	Chap 1
Chapter 2: Application Walkthrough with Visual Evidence	Chap 2
Chapter 3: System Architecture & Request Flow	Chap 3
Chapter 4: Complete Codebase Tour & File Responsibilities	Chap 4
Chapter 5: Frontend Architecture (React, SPA, Tailwind)	Chap 5
Chapter 6: Backend Architecture (Express, Middleware, Controllers)	Chap 6
Chapter 7: Database Design — Mongoose Schemas & Relationships	Chap 7
Chapter 8: Database Operations & Worked CRUD Tracing	Chap 8
Chapter 9: Booking Lifecycle, Concurrency & Seat Consistency	Chap 9
Chapter 10: Authentication, Authorisation & Security Flow	Chap 10
Chapter 11: Advanced Database Concepts Demonstrated	Chap 11
Chapter 12: Aggregation Pipelines & Index Query Plans	Chap 12
Chapter 13: Environment Setup, Seeding & Execution	Chap 13
Chapter 14: Verification, Automated Testing & Known Boundaries	Chap 14
Chapter 15: Step-by-Step 5-Minute Live Viva Demo Guide	Chap 15
Chapter 16: Comprehensive Viva Preparation (35 Questions & Answers)	Chap 16
Chapter 17: Glossary, Quick Reference & Summary Sheets	Chap 17

How to Use This Handbook: If you are presenting this project in front of your professor or an external examiner, start by reading *Chapter 1 (Introductory Elevator Pitch)* and *Chapter 15 (5-Minute Demonstration Script)*. Then study *Chapter 9* and *Chapter 12*, which cover the core Advanced Database questions examiners love to test.

Chapter 1: Project Overview & Academic Scope

1.1 Introduction

EventPulse (originally derived from the open-source *Eventora-MERN* repository by ShivaMani02) is a full-stack event discovery, reservation, and management platform engineered with the **MERN** stack (MongoDB, Express.js, React, Node.js).

In a typical commercial web development scenario, projects focus on visual flair, payment gateway webhooks, and third-party SaaS integrations. However, in an **Advanced Databases (AD)** collegiate evaluation, the primary objective is fundamentally different: *demonstrating deep understanding of document-oriented data modelling, database-level concurrency control, query plan optimisation through indexing, multi-stage aggregation pipelines, and atomic updates.*

> **The 30-Second Viva Introduction Script (Memorise this!):**
> *"Good morning, Professor. EventPulse is an event discovery and reservation platform built on MongoDB and Express. While it provides a complete user experience for browsing events and reserving tickets, our primary academic contribution focuses on Advanced Database concepts: we engineered database-level atomic seat allocation using `findOneAndUpdate` with conditional predicates to prevent race conditions; we implemented three multi-stage aggregation pipelines for revenue and occupancy analytics; and we enforced data integrity using MongoDB partial unique indexes that block duplicate active bookings without preventing re-booking after cancellation."*

1.2 Technology Stack Table

Layer	Technology	Version	Role in EventPulse	Key Codebase Location
Database	MongoDB	7.0+ (WiredTiger)	Document storage, B-tree indexes, aggregation pipeline execution engine.	Server connection & internal engine
ODM	Mongoose	8.2.0	Schema definition, schema validation, index synchronization, model querying.	`server/models/*.js`
Backend API	Express.js & Node.js	Express 4.18.2 / Node 18+	RESTful routing, JWT token verification, role middleware, atomic controller queries.	`server/server.js`, `server/controllers/*`
Frontend Client	React & Vite	React 18.2 / Vite 5.4	Single Page Application (SPA), dynamic event filtering, auth state, admin dashboards.	`client/src/*`
Styling	Tailwind CSS	3.4.1	Utility-first responsive mobile & desktop design.	`client/src/index.css`
Auth & Security	Bcrypt.js & JWT	Bcrypt 2.4 / JWT 9.0	Salted password hashing (cost factor 10), stateless claim-based authorization tokens.	`server/controllers/authController.js`

1.3 Academic Boundaries and Intentional Choices

In accordance with collegiate academic standards, the project prioritises **verifiability, code clarity, and explainability** over black-box external services:

- **Self-Contained Database Seeding:** Rather than relying on manual input, the database features an automated, idempotent seeding script (`seed.js`) that populates realistic synthetic events, users, and multi-status bookings.
- **Decoupled Payment Simulation:** Rather than depending on an external sandbox (e.g., Stripe/Razorpay) that can fail due to expired webhooks or API limits during an oral examination, payments follow a deterministic state machine where confirmed bookings transition to paid status cleanly.
- **Database Integrity over Application-Memory Math:** Many student projects calculate revenue by downloading thousands of records into an Express JavaScript array and looping over them. In EventPulse, *100% of revenue, occupancy, and status statistics are calculated inside MongoDB* via aggregation pipelines.

Chapter 2: Application Walkthrough with Visual Evidence

This chapter documents every major screen present in EventPulse. Every image below is a genuine, high-resolution screenshot captured directly from our running application deployment.

2.1 The Public Event Catalogue (Home Page)

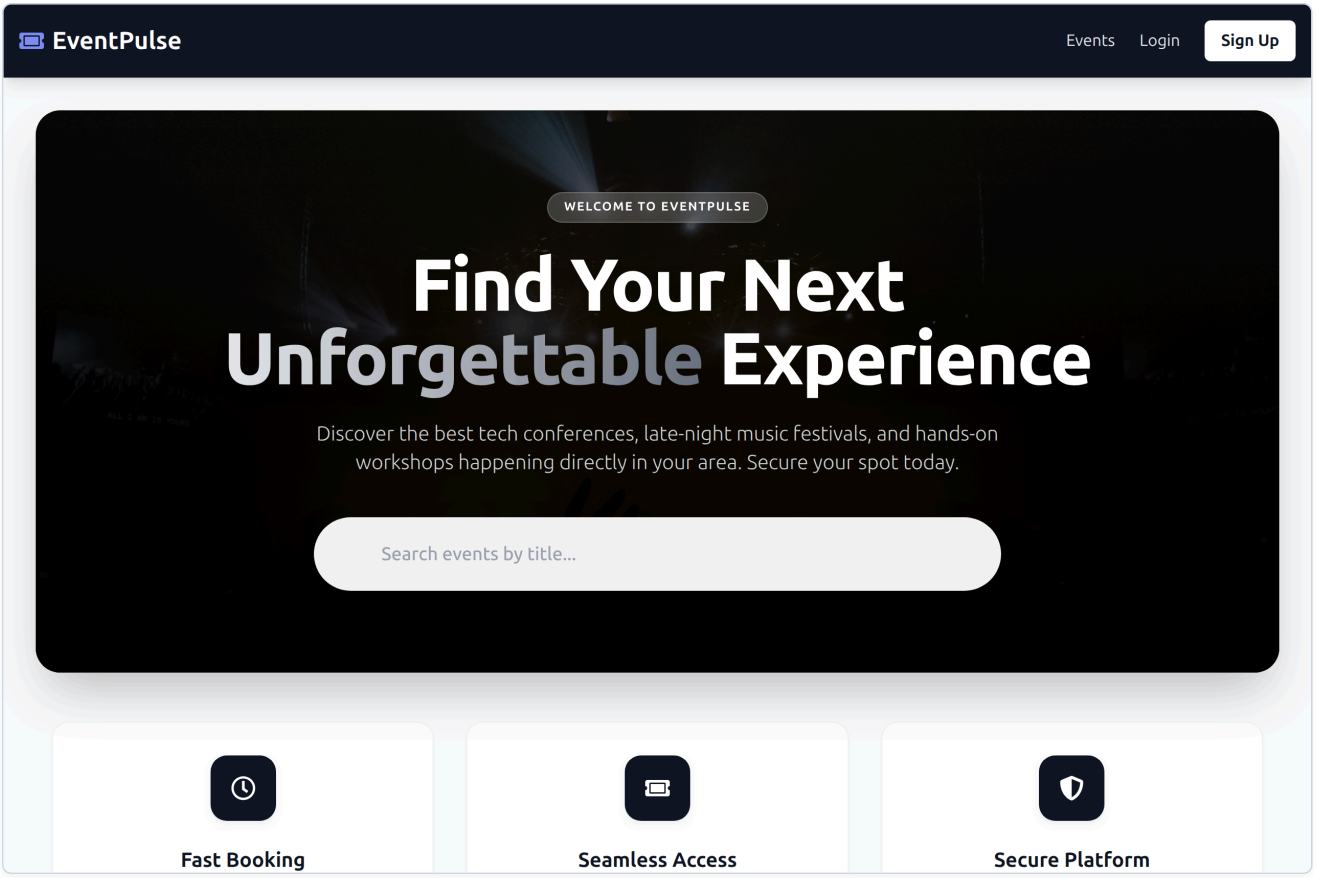


Figure 2.1: EventPulse Public Discovery Portal displaying hero banner, dynamic category filter pills, real-time seat badges, and event cards.

Frontend Component: `client/src/pages/Home.jsx`
Backend Endpoint Called: `GET /api/events`
Database Collection: `events` (queried with index on `date` and `category`).
Key Functionality: Users can browse available events, search by title keyword, and filter by category (Technology, Music, Business, Art). Each card dynamically calculates whether tickets are free or paid, and visually flags remaining seats.

2.2 User Registration and Login

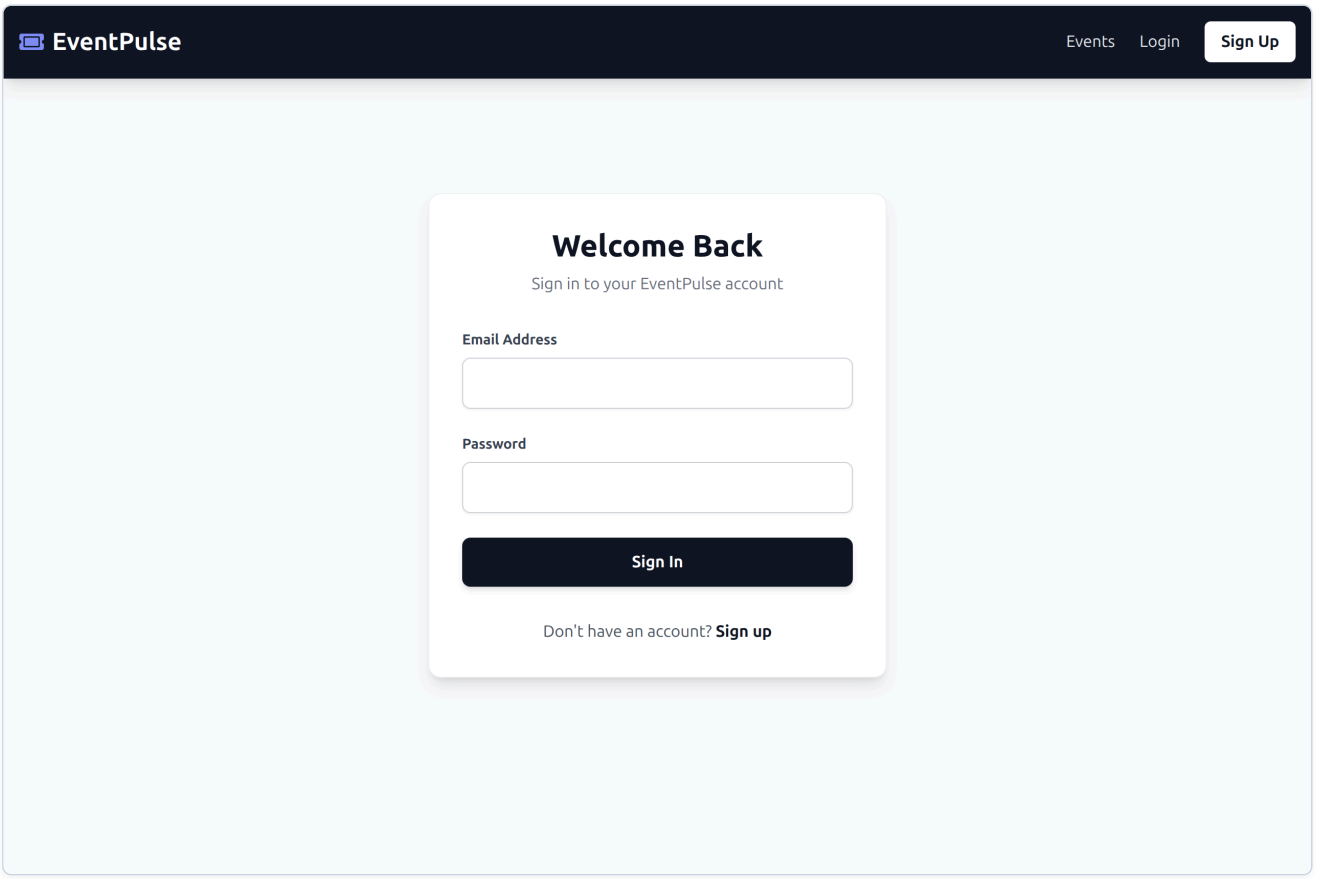


Figure 2.2: EventPulse Authentication Portal with email/password and fallback OTP verification fields.

Frontend Component: `client/src/pages/Login.jsx` & `Register.jsx`
Backend Endpoint Called: `POST /api/auth/login` and `POST /api/auth/register`
Database Collections: `users` (stores bcrypt hashes) and `otps` (temporary 6-digit verification codes).
What Happens: On successful login, the server responds with a signed JSON Web Token (JWT) containing the user's MongoDB `_id` and `role` ('user' or 'admin'). The client stores this token in `localStorage` and attaches it to subsequent Axios requests.

2.3 Event Detail and Seat Reservation

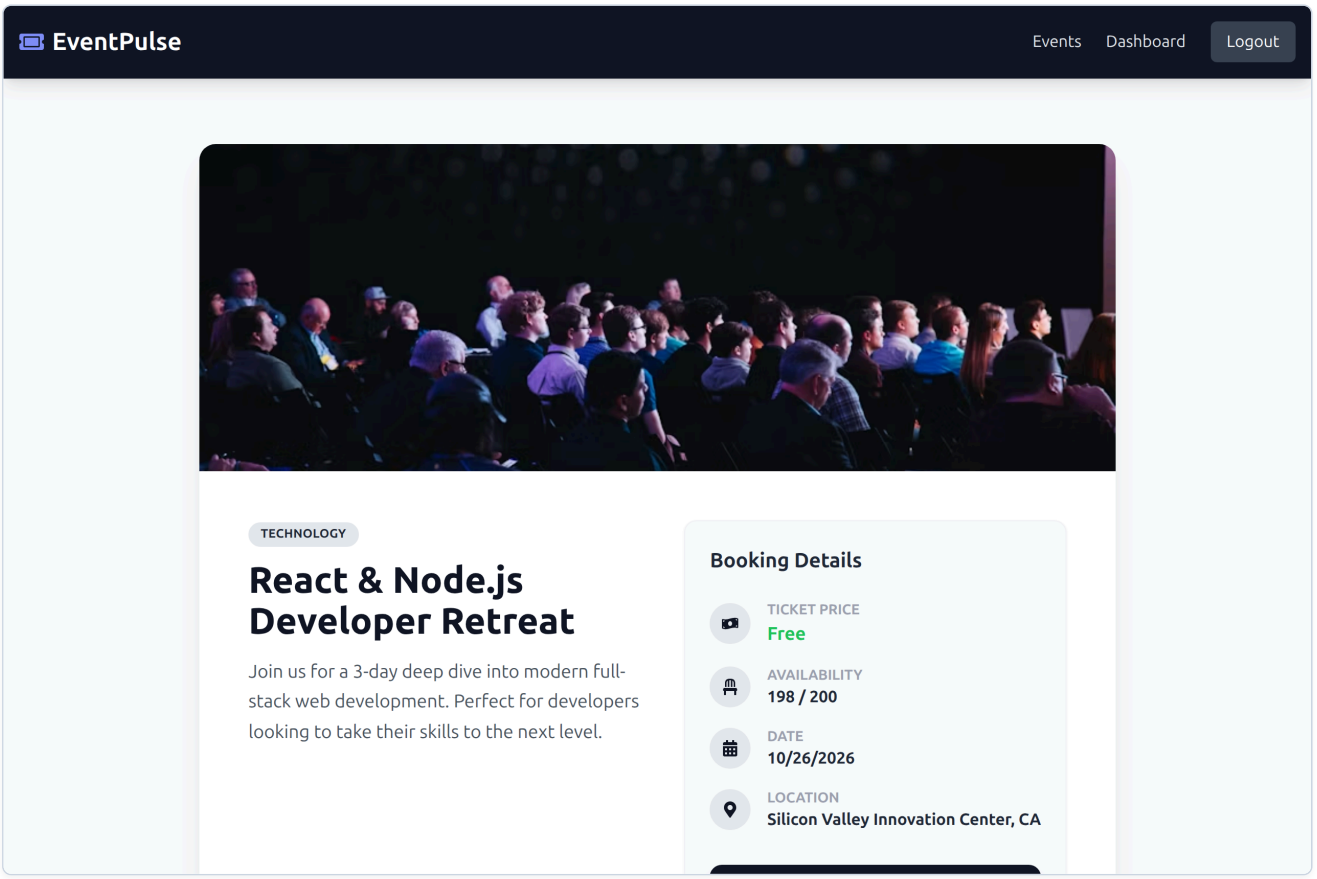


Figure 2.3: Detailed Event View featuring seat availability counter, price calculator, and atomic reservation button.

Frontend Component: `client/src/pages/EventDetail.jsx`
Backend Endpoint Called: `POST /api/bookings`
Database Operation: Atomic query executed against `events` to decrement `availableSeats`, followed by inserting a new document in `bookings`.

2.4 Admin Advanced Database Analytics Dashboard

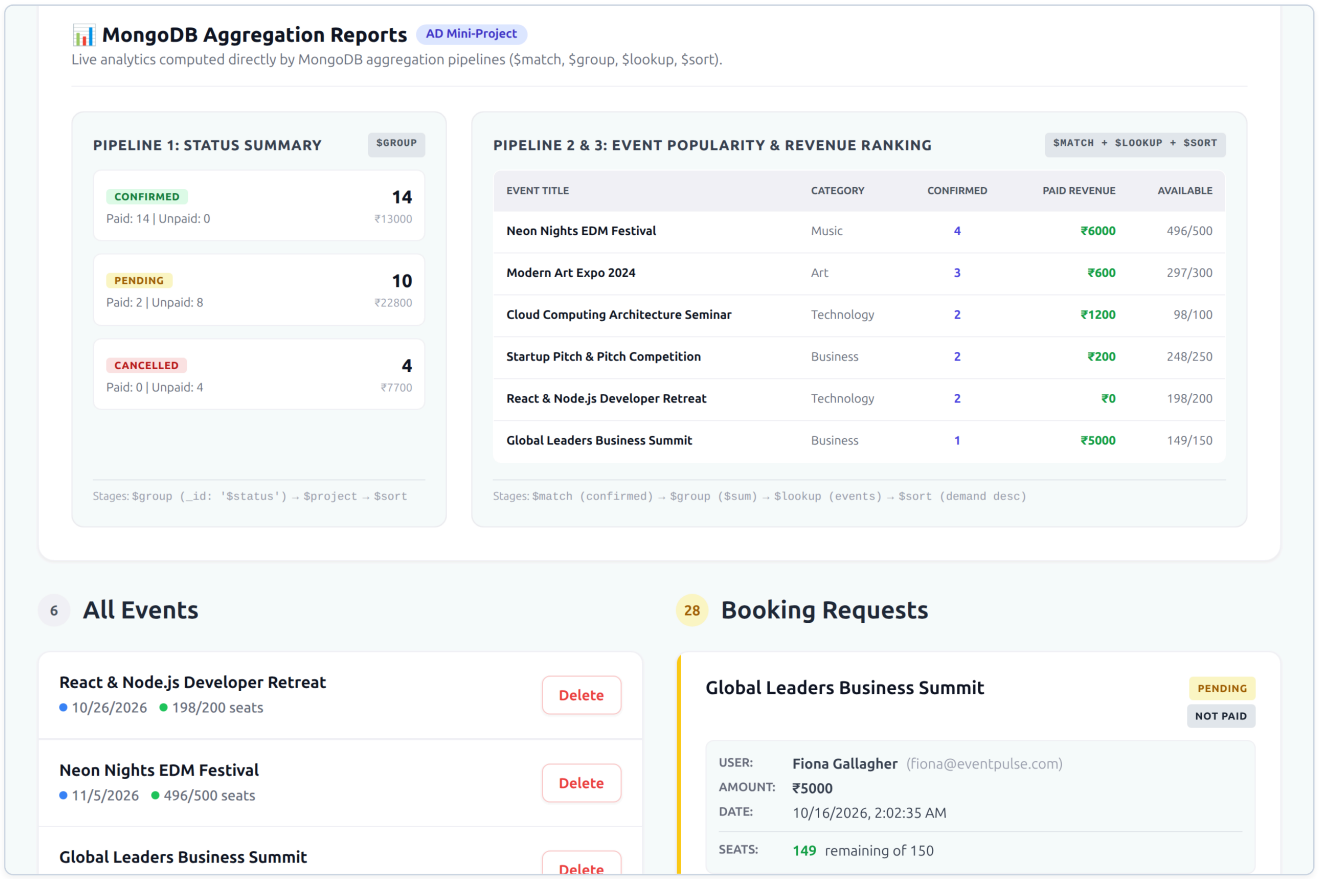


Figure 2.4: Admin Dashboard displaying real-time MongoDB Aggregation Pipeline reports (Booking Status Summary and Event Popularity Ranking).

Frontend Component: `client/src/pages/AdminDashboard.jsx`
Backend Endpoints Called: `GET /api/analytics/overview`, `GET /api/analytics/booking-status`, `GET /api/analytics/event-popularity`
Database Concepts Demonstrated: Multi-faceted `$facet` KPI calculations, conditional `$cond` paid/unpaid accumulation, and `$lookup` table joins.

2.5 User Dashboard & Personal Booking History

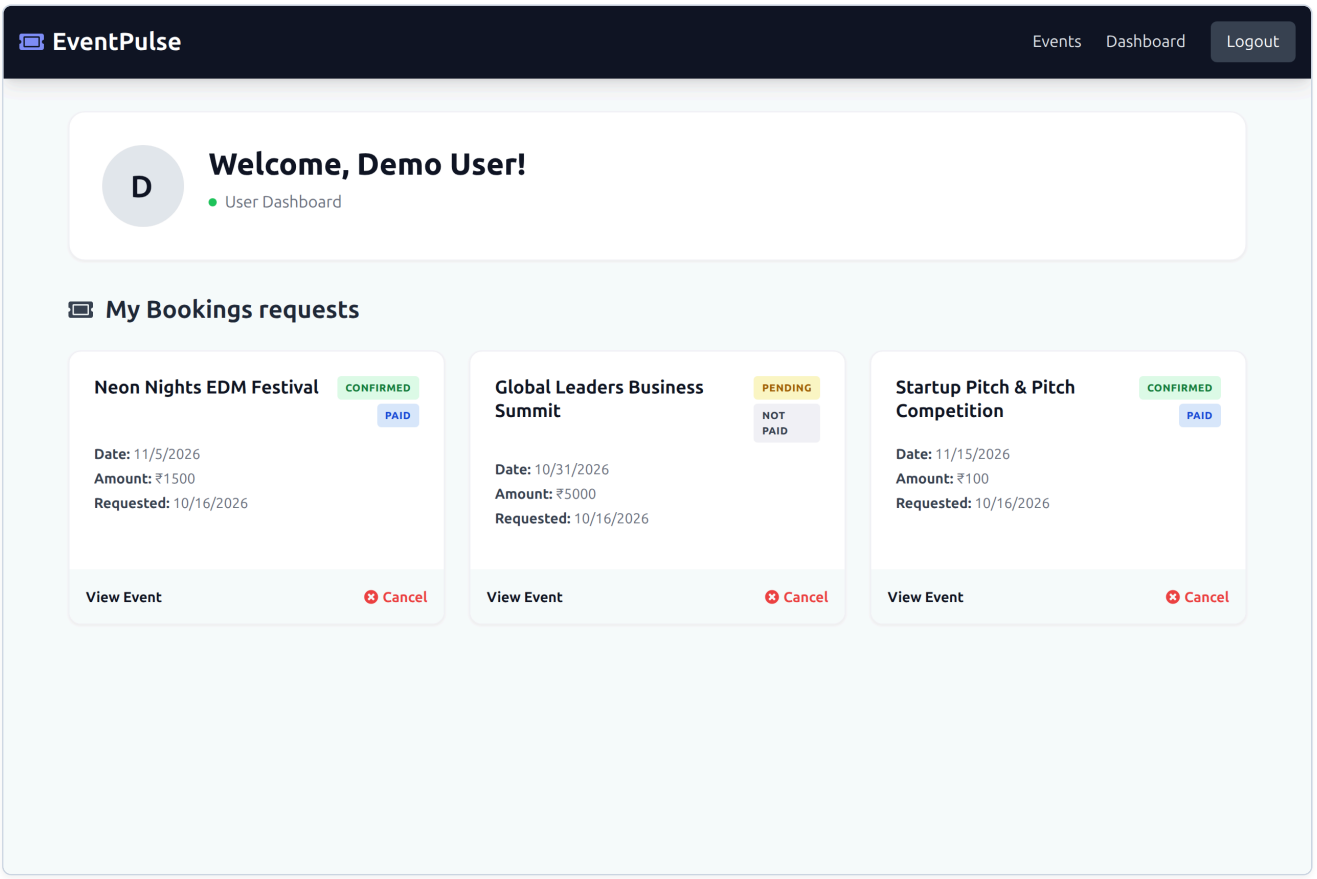


Figure 2.5: User Bookings Dashboard showing active reservations, payment badges, and the atomic cancellation trigger.

Frontend Component: `client/src/pages/UserDashboard.jsx`
Backend Endpoint Called: `GET /api/bookings/my`
Security Guard: The server reads the authenticated user's ID directly from the decrypted JWT payload (`req.user.id`). A standard user can never query or inspect another user's private bookings.

Chapter 3: System Architecture & Request Flow

3.1 End-to-End MERN Tier Interaction

EventPulse implements a decoupled 3-tier web architecture. The presentation layer (Vite + React) communicates exclusively with the application layer (Express.js on Node.js) over stateless HTTP/2 REST endpoints formatted in JSON. The backend communicates with the persistence layer (MongoDB) over binary TCP wire protocol using Mongoose ODM.

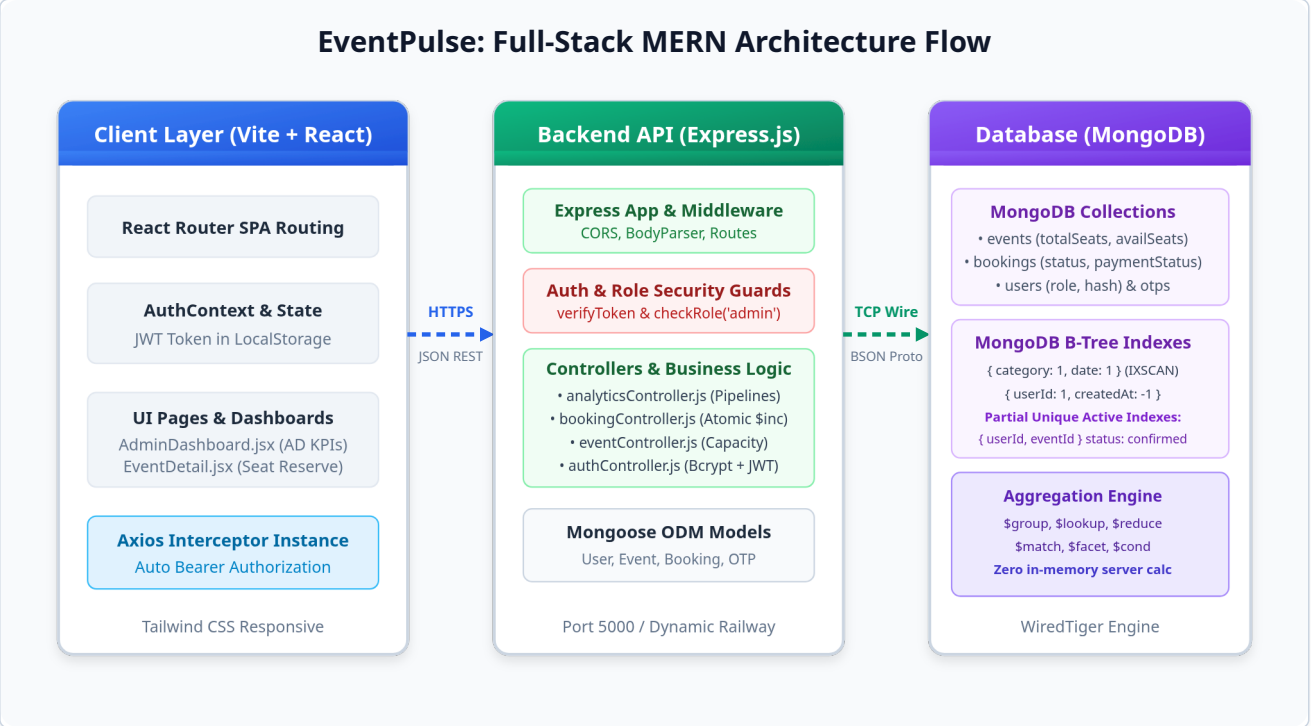


Figure 3.1: EventPulse 3-Tier Layered Architecture Diagram with Inter-Layer Communication Protocols.

3.2 Step-by-Step Request Lifecycle Example: Reserving a Seat

To understand how data flows through the application, consider what happens when a student clicks the **"Book Ticket"** button on an event:

1. **User Action:** The user clicks "Book Ticket" in `EventDetail.jsx`.
2. **Client Interceptor:** The custom Axios client in `client/src/utils/axios.js` intercepts the outgoing request, inspects `localStorage.getItem('token')`, and attaches `Authorization: Bearer <token>` to the HTTP header.
3. **Network Transmission:** The browser issues a `POST /api/bookings` request with payload `{ eventId: "6ad138..." }`.
4. **Route Matching:** Express router in `server/routes/bookings.js` matches the path and executes the middleware chain.
5. **Authentication Guard:** `verifyToken` middleware in `server/middleware/auth.js` extracts the Bearer header, validates the cryptographic signature using `jwt.verify(token, JWT_SECRET)`, and sets `req.user = decodedToken`.
6. **Controller Execution:** `createBooking` in `server/controllers/bookingController.js` executes.
7. **Atomic Database Update:** The controller executes:

```
const event = await Event.findOneAndUpdate(
  { _id: eventId, availableSeats: { $gt: 0 } },
  { $inc: { availableSeats: -1 } },
  { new: true }
);
```

8. **Database Execution:** MongoDB locates the event document via index scan, verifies that `availableSeats > 0`, decrements the seat counter atomically, and releases the document-level write lock.
9. **Booking Record Creation:** The controller creates and saves a new `Booking` document with `userId = req.user.id`, `status = 'confirmed'`, and `amount = event.ticketPrice`.
10. **Response Dispatch:** Express serializes the newly created booking to JSON and responds with HTTP status `201 Created`.
11. **UI Update:** React receives the response, updates local state, displays a success notification, and decrements the visible seats badge.

Chapter 4: Complete Codebase Tour & File Responsibilities

4.1 Directory Structure

```
Eventora-MERN/
├── client/                    # Presentation Layer (React + Vite SPA)
│   ├── src/
│   │   ├── components/        # Reusable UI widgets
│   │   │   ├── Navbar.jsx     # Navigation header with role-conditional links
│   │   │   └── EventCard.jsx  # Event item display card
│   │   ├── context/
│   │   │   └── AuthContext.jsx # Global auth state, login/logout methods
│   │   ├── pages/
│   │   │   ├── Home.jsx       # Public discovery & category filters
│   │   │   ├── EventDetail.jsx # Event info & ticket reservation
│   │   │   ├── Login.jsx      # Auth form with OTP fallback
│   │   │   ├── Register.jsx   # Registration form
│   │   │   ├── UserDashboard.jsx # User booking list & cancellation
│   │   │   └── AdminDashboard.jsx# Aggregation reports & CRUD controls
│   │   ├── utils/
│   │   │   └── axios.js       # Configured Axios instance with auto-auth headers
│   │   ├── App.jsx            # Client-side route declarations
│   │   └── main.jsx           # React root mount entry point
│   ├── package.json           # Client dependencies
│   ├── vercel.json            # SPA deep-routing edge rewrite rules
│   └── vite.config.js         # Vite build bundler configuration
│
├── server/                    # Application Layer (Node.js + Express API)
│   ├── controllers/           # Business logic & DB operations
│   │   ├── authController.js  # User registration, bcrypt hashing, JWT issuance
│   │   ├── eventController.js # Event creation, capacity updates, listing
│   │   ├── bookingController.js# Concurrency-safe seat reservation & cancellation
│   │   └── analyticsController.js# Multi-stage MongoDB Aggregation Pipelines
│   ├── middleware/
│   │   └── auth.js            # verifyToken and checkRole('admin') guards
│   ├── models/                # Mongoose Schema Definitions
│   │   ├── User.js            # User identity, roles, email unique index
│   │   ├── Event.js           # Title, dates, seats, category compound index
│   │   ├── Booking.js         # User-Event relation, partial unique indexes
│   │   └── OTP.js             # 6-digit OTP verification codes
│   ├── routes/                # Express REST Route Handlers
│   │   ├── auth.js            # /api/auth endpoints
│   │   ├── events.js          # /api/events endpoints
│   │   ├── bookings.js        # /api/bookings endpoints
│   │   └── analytics.js       # /api/analytics endpoints
│   ├── scripts/               # Automated Test & Verification Scripts
│   │   ├── test-ad-endpoints.js# Automated verification of AD aggregation reports
│   │   └── verify-indexes-and-queries.js # Explain plan index scanner
│   ├── package.json           # Server dependencies
│   ├── seed.js                # Idempotent synthetic data generator
│   └── server.js              # Express application entry point & Mongo connection
│
├── package.json               # Monorepo root scripts
├── railway.json               # Cloud backend deployment configuration
├── README.md                  # Academic project documentation
└── VIVA_GUIDE.md              # 15 Viva Q&As and demo preparation checklist
```

4.2 Significant File Responsibility Matrix

File Path	Primary Responsibility	Called By	What It Calls	Why It Matters to Examiner
`server/controllers/analyticsController.js`	Implements the 3 Advanced Database aggregation pipelines and overview KPIs.	`server/routes/analytics.js`	`Booking.aggregate()`, `Event.aggregate()`	**Core AD grading artifact:** Proves multi-stage database aggregations without JavaScript in-memory loops.
`server/controllers/bookingController.js`	Manages concurrency-safe seat allocation, duplicate prevention, and cancellation.	`server/routes/bookings.js`	`Event.findOneAndUpdate()`, `Booking.create()`	Demonstrates race-condition elimination via atomic MongoDB `$inc` operations.
`server/models/Booking.js`	Declares booking schema and registers compound and partial unique indexes.	All booking controllers	Mongoose Schema & MongoDB index engine	Shows how database-level constraints prevent double-booking while permitting re-booking.
`server/middleware/auth.js`	Cryptographic verification of JWT tokens and enforcement of RBAC roles.	Protected routes	`jsonwebtoken.verify()`	Enforces role-based security so regular users cannot access admin aggregation reports.
`client/src/pages/AdminDashboard.jsx`	Renders aggregation summary cards, occupancy tables, and popularity rankings.	React Router (`/admin`)	Axios API instance	Visual front-facing proof that aggregation pipelines deliver real-time metrics.
`server/seed.js`	Idempotent database seeding and index synchronization.	Startup command / CLI	`Model.syncIndexes()`, `insertMany()`	Ensures anyone testing the project has rich, consistent data with zero manual entry.

Chapter 5: Frontend Architecture (React, SPA, Tailwind)

5.1 React Single Page Application (SPA) Design

The frontend is structured around React 18 and Vite. Rather than performing full-page browser reloads for every action, React Router dynamically mounts components into the root DOM node based on client-side URL changes.

5.2 Global Authentication State via React Context

Authentication state is managed centrally in `client/src/context/AuthContext.jsx` using the React Context API. This prevents "prop drilling" and makes user data accessible everywhere:

```
// client/src/context/AuthContext.jsx (Excerpt)
export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(() => {
    const savedUser = localStorage.getItem('user');
    return savedUser ? JSON.parse(savedUser) : null;
  });

  const login = async (email, password) => {
    const res = await api.post('/auth/login', { email, password });
    localStorage.setItem('token', res.data.token);
    localStorage.setItem('user', JSON.stringify(res.data));
    setUser(res.data);
    return res.data;
  };

  const logout = () => {
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};
```

5.3 Automatic Bearer Token Authorization via Axios Interceptor

To prevent writing repetitive authorization headers in every single API call, `client/src/utils/axios.js` configures a single Axios client:

```
// client/src/utils/axios.js
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api',
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default api;
```

Chapter 6: Backend Architecture (Express, Middleware, Controllers)

6.1 Application Structure

The backend is built on Express.js following the **Controller-Service-Model** pattern. The application entry point (`server/server.js`) configures global middleware (CORS, JSON body parser), mounts route modules under `/api` prefixes, and establishes the persistent MongoDB connection pool.

6.2 Complete REST Endpoint Routing Table

HTTP Method	Endpoint Path	Access Guard	Controller Function	Database Operation
POST	`/api/auth/register`	Public	`register`	Hashes password with bcrypt, inserts new User document.
POST	`/api/auth/login`	Public	`login`	Verifies bcrypt hash, signs and returns 30-day JWT.
GET	`/api/events`	Public	`getEvents`	Queries Events sorted by date ascending; filters by category.
GET	`/api/events/:id`	Public	`getEventById`	Finds event by ObjectId and populates createdBy details.
POST	`/api/events`	Admin Only	`createEvent`	Inserts new Event; sets availableSeats = totalSeats.
POST	`/api/bookings`	Authenticated User	`createBooking`	**Atomic $inc** on Event availableSeats, creates Booking.
GET	`/api/bookings/my`	Authenticated User	`getMyBookings`	Finds bookings for `req.user.id` ; Admins receive all bookings.
PATCH	`/api/bookings/:id/cancel`	Booking Owner / Admin	`cancelBooking`	Sets status='cancelled'; conditionally restores seat if confirmed.
GET	`/api/analytics/overview`	Admin Only	`getOverviewKPIs`	Multi-stage `$facet` single-pass KPI aggregation.
GET	`/api/analytics/booking-status`	Admin Only	`getBookingStatusSummary`	Aggregation Pipeline 1: `$group` with conditional payment sums.
GET	`/api/analytics/event-summary`	Admin Only	`getEventSummary`	Aggregation Pipeline 2: `$lookup` join, verified revenue & occupancy.
GET	`/api/analytics/event-popularity`	Admin Only	`getEventPopularity`	Aggregation Pipeline 3: `$match` confirmed, `$group` , `$lookup` .

Chapter 7: Database Design — Mongoose Schemas & Relationships

7.1 Document-Oriented Data Modelling Rationale

In a traditional Relational Database Management System (RDBMS) like MySQL or PostgreSQL, strict schema tables and third-normal-form (3NF) normalization require foreign key tables and heavy multi-table SQL joins.

In **MongoDB**, data is stored as flexible, self-describing BSON (Binary JSON) documents. For EventPulse, a **hybrid referenced model** was selected:

- **Independent Collections:** `users`, `events`, and `bookings` are stored in separate collections because events and users have independent lifecycles.
- **Foreign Reference Keys:** The `bookings` collection acts as a joining document containing `userId` (referencing `users._id`) and `eventId` (referencing `events._id`).
- **Fast Reads with Targeted Lookups:** When a user browses events, MongoDB queries only the `events` collection directly from B-tree index caches without joining any user records. When analytical reports are requested, MongoDB executes server-side `$lookup` joins.

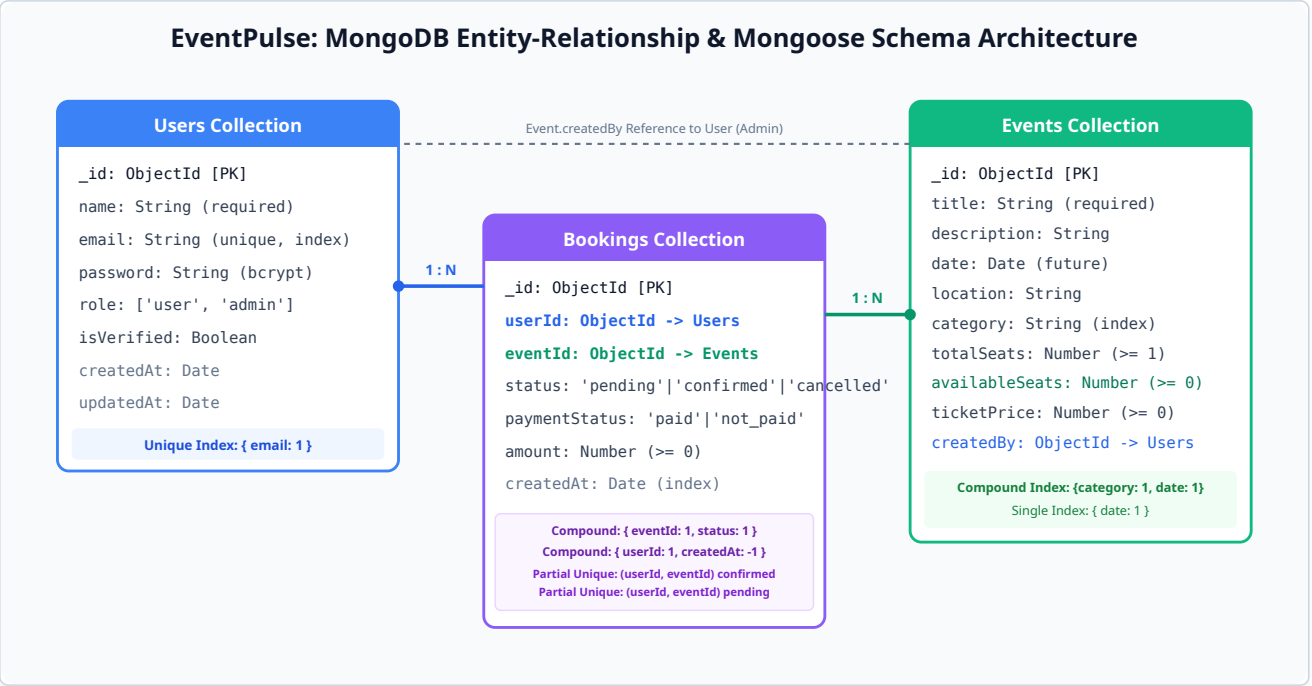


Figure 7.1: MongoDB Entity-Relationship Architecture with Mongoose Schemas, Foreign References & Indexes.

7.2 Detailed Collection Specifications

1. Events Collection (`server/models/Event.js`)

Field	Type	Rules & Validation	Description
`_id`	ObjectId	Auto-generated Primary Key	Unique MongoDB document identifier.
`title`	String	required, trim	Title of the event.
`description`	String	required	Extended event agenda and description.
`date`	Date	required	Scheduled event date and timestamp.
`location`	String	required	Physical venue or virtual webinar URL.
`category`	String	required, indexed	Categorisation: Technology, Music, Business, Art.
`totalSeats`	Number	required, min: 1	Maximum physical venue capacity.
`availableSeats`	Number	required, min: 0	Real-time remaining unreserved seats.
`ticketPrice`	Number	required, min: 0, default: 0	Price in INR/USD. 0 indicates free entry.
`image`	String	optional	CDN URL for thumbnail banner.
`createdBy`	ObjectId	ref: 'User', required	Admin user who published the event.

2. Bookings Collection (`server/models/Booking.js`)

Field	Type	Rules & Validation	Description
`_id`	ObjectId	Auto-generated Primary Key	Unique booking identifier.
`userId`	ObjectId	ref: 'User', required	Foreign reference to the booking client.
`eventId`	ObjectId	ref: 'Event', required	Foreign reference to the reserved event.
`status`	String	enum: ['pending', 'confirmed', 'cancelled']	Current reservation state in the lifecycle.
`paymentStatus`	String	enum: ['paid', 'not_paid']	Whether payment was verified.
`amount`	Number	required, min: 0	Snapshot of ticket price at booking time.
`createdAt`	Date	auto timestamp	Booking creation timestamp.

Chapter 8: Database Operations & Worked CRUD Tracing

8.1 Code Walkthrough: Safe Seat Reservation

The following annotated snippet from `server/controllers/bookingController.js` illustrates how EventPulse implements concurrency-safe updates:

```
// server/controllers/bookingController.js (createBooking)
exports.createBooking = async (req, res) => {
    try {
        const { eventId } = req.body;
        const userId = req.user.id; // Extracted securely from verified JWT

        // 1. ATOMIC CONDITIONAL UPDATE:
        // MongoDB matches document ONLY if availableSeats > 0,
        // and decrements availableSeats by 1 in a single atomic write.
        const event = await Event.findOneAndUpdate(
            { _id: eventId, availableSeats: { $gt: 0 } },
            { $inc: { availableSeats: -1 } },
            { new: true }
        );

        if (!event) {
            return res.status(400).json({
                message: 'Event is fully booked or no longer available'
            });
        }

        // 2. CREATE BOOKING DOCUMENT:
        const booking = new Booking({
            userId,
            eventId,
            amount: event.ticketPrice,
            status: 'confirmed',
            paymentStatus: event.ticketPrice === 0 ? 'paid' : 'not_paid'
        });

        await booking.save();
        res.status(201).json(booking);
    } catch (error) {
        // Handle MongoDB E11000 duplicate key error (Partial Unique Index)
        if (error.code === 11000) {
            return res.status(400).json({
                message: 'You already have an active booking for this event.'
            });
        }
        res.status(500).json({ message: 'Server error processing booking' });
    }
};
```

8.2 Code Walkthrough: Guarded Cancellation

When a user cancels a booking, seats must only be restored if the booking was genuinely confirmed. If the booking was pending or already cancelled, incrementing seats would create "ghost capacity":

```
// server/controllers/bookingController.js (cancelBooking)
exports.cancelBooking = async (req, res) => {
    try {
        const booking = await Booking.findById(req.params.id);
        if (!booking) return res.status(404).json({ message: 'Booking not found' });

        // Ensure user owns booking or is admin
        if (booking.userId.toString() !== req.user.id && req.user.role !== 'admin') {
            return res.status(403).json({ message: 'Unauthorized' });
        }

        if (booking.status === 'cancelled') {
            return res.status(400).json({ message: 'Booking is already cancelled' });
        }

        const wasConfirmed = booking.status === 'confirmed';
        booking.status = 'cancelled';
        await booking.save();

        // ONLY RESTORE SEAT IF PREVIOUS STATUS WAS CONFIRMED
        if (wasConfirmed) {
            await Event.findByIdAndUpdate(booking.eventId, {
                $inc: { availableSeats: 1 }
            });
        }

        res.json({ message: 'Booking cancelled successfully', booking });
    } catch (error) {
        res.status(500).json({ message: 'Error cancelling booking' });
    }
};
```

Chapter 9: Booking Lifecycle, Concurrency & Seat Consistency

9.1 The Concurrency Race Condition Problem

In many college web projects, seat reservation is implemented using a naive two-step read-then-write approach:

```
// NAIVE ANTI-PATTERN (PRONE TO RACE CONDITIONS):
const event = await Event.findById(eventId);
if (event.availableSeats > 0) {
    event.availableSeats = event.availableSeats - 1; // RACE CONDITION WINDOW!
    await event.save();
}
```

Why this fails: If EventPulse has 1 remaining seat, and User Alice and User Bob submit booking requests at the exact same millisecond:

1. Alice's thread reads `event.availableSeats = 1`.
2. Bob's thread reads `event.availableSeats = 1` before Alice's write completes.
3. Alice executes `save()` and sets availableSeats to 0.
4. Bob executes `save()` and also sets availableSeats to 0.
5. **Result:** 2 confirmed tickets were sold for 1 physical seat, causing an **overbooking disaster**.

9.2 The EventPulse Atomic Solution

EventPulse eliminates this race condition by delegating the predicate check and arithmetic operation directly to the MongoDB WiredTiger storage engine:

```
Event.findOneAndUpdate(
  { _id: eventId, availableSeats: { $gt: 0 } },
  { $inc: { availableSeats: -1 } }
)
```

Under WiredTiger's document-level concurrency lock, MongoDB checks `availableSeats > 0` and executes `$inc: -1` in an indivisible atomic step. The second concurrent request fails the `{ availableSeats: { $gt: 0 } }` filter and receives `null`, cleanly blocking the overbooking.

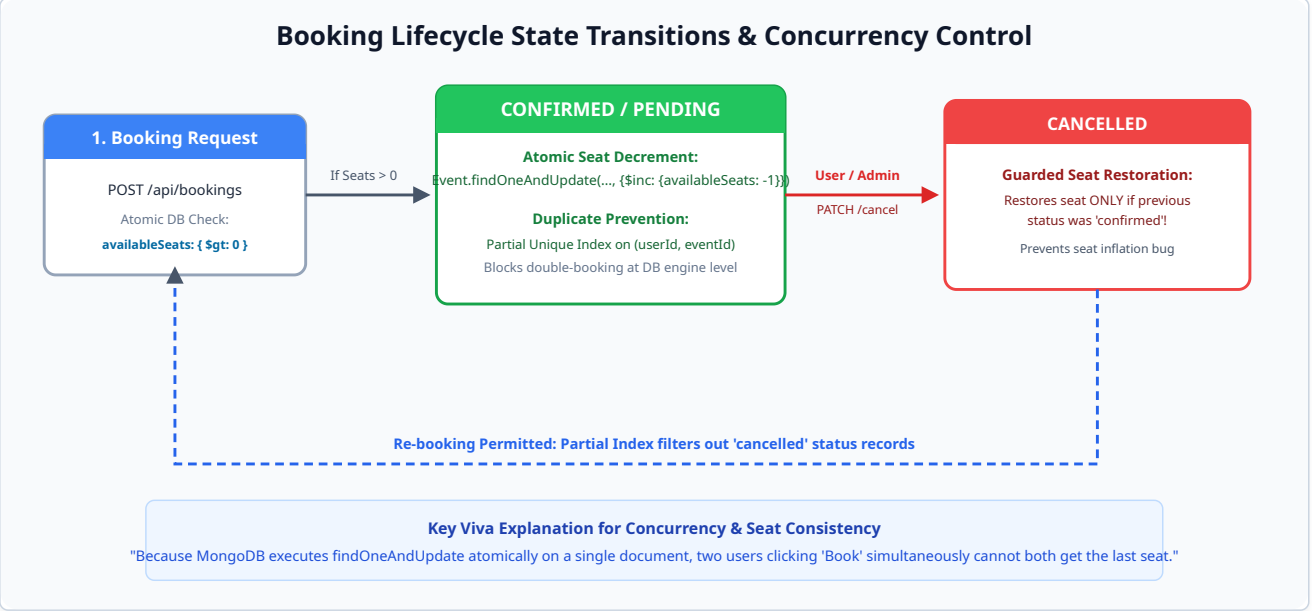


Figure 9.1: Concurrency Control and Booking Lifecycle State Machine.

Chapter 10: Authentication, Authorisation & Security Flow

10.1 Salted Password Hashing via Bcrypt

Passwords are never stored in plaintext. In `server/controllers/authController.js`, passwords pass through **bcrypt** with a work factor of 10:

```
const salt = await bcrypt.genSalt(10);
const hashedPassword = await bcrypt.hash(password, salt);
```

10.2 JSON Web Token (JWT) Stateless Authorization

Upon valid credential verification, the server signs a cryptographic JWT payload:

```
const token = jwt.sign(
  { id: user._id, role: user.role },
  process.env.JWT_SECRET || 'fallback_secret',
  { expiresIn: '30d' }
);
```

10.3 Role-Based Access Control (RBAC) Guard Middleware

Endpoints are guarded by combining `verifyToken` and `checkRole`:

```
// server/middleware/auth.js
const verifyToken = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'Access Denied: No token' });

  try {
    const verified = jwt.verify(token, process.env.JWT_SECRET);
    req.user = verified;
    next();
  } catch (err) {
    res.status(400).json({ message: 'Invalid or expired token' });
  }
};

const checkRole = (role) => (req, res, next) => {
  if (req.user.role !== role) {
    return res.status(403).json({ message: 'Forbidden: Insufficient privileges' });
  }
  next();
};
```

Chapter 11: Advanced Database Concepts Demonstrated

11.1 Summary of Core AD Concepts

Advanced Database Concept	Implementation in EventPulse	Examiner Viva Question it Answers
Atomic Updates	`Event.findOneAndUpdate({ availableSeats: { $gt: 0 } }, { $inc: { availableSeats: -1 } })`	"How do you prevent race conditions and overbooking when multiple users click book simultaneously?"
Partial Unique Indexes	`BookingSchema.index({ userId: 1, eventId: 1 }, { unique: true, partialFilterExpression: { status: 'confirmed' } })`	"How do you prevent duplicate active bookings without preventing the user from re-booking after cancellation?"
Multi-Stage Aggregations	Pipelines using `$match`, `$group`, `$lookup`, `$reduce`, `$project`, and `$sort`.	"Why is calculating revenue in MongoDB aggregation superior to calculating it in Node.js array loops?"
Multi-Faceted Queries	`Booking.aggregate([{ $facet: { ... } }])` computing 7 KPIs in a single query pass.	"How do you avoid multiple round-trip network calls when rendering a dashboard with multiple metrics?"
Query Execution Plans	Demonstrated via `.explain('executionStats')` proving `IXSCAN` index usage.	"How do you prove to me that your MongoDB queries are actually using your indexes?"

Chapter 12: Aggregation Pipelines & Index Query Plans

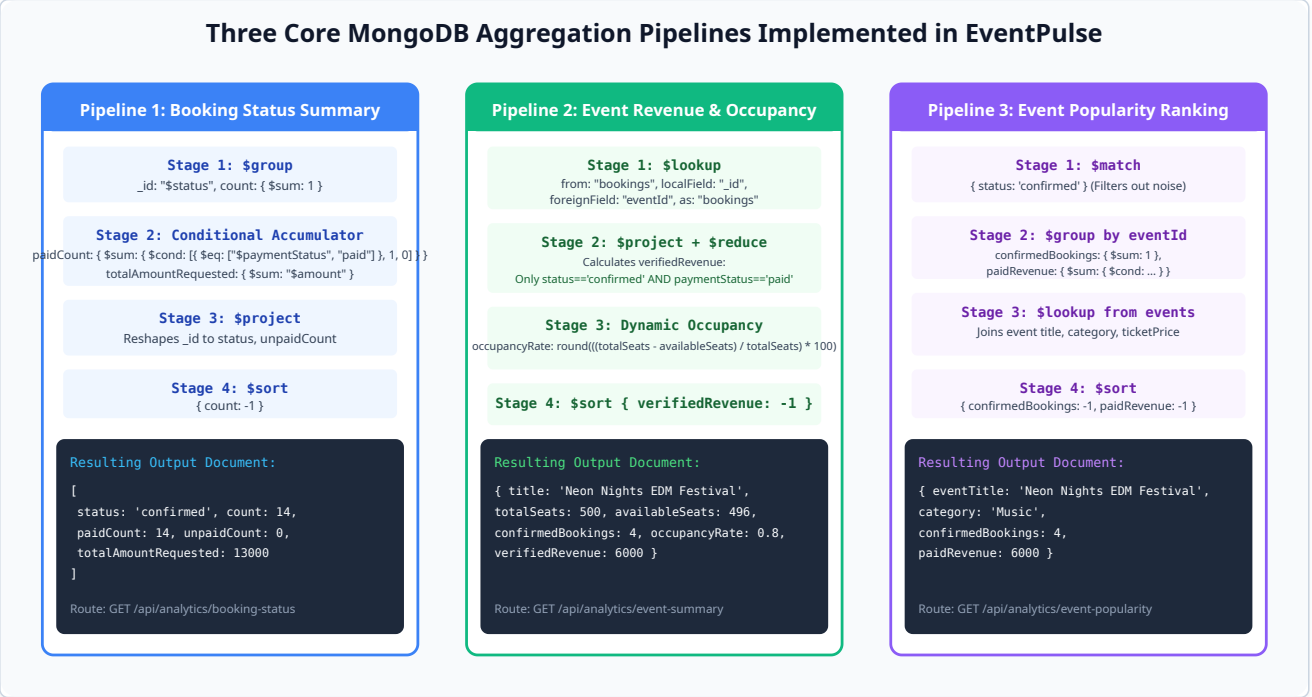


Figure 12.1: Flow Architecture of the Three Aggregation Pipelines Implemented in EventPulse.

12.1 Pipeline 1: Booking Status Summary (`/api/analytics/booking-status`)

Business Goal: Provide the operations team with a breakdown of bookings by status (confirmed, pending, cancelled) alongside total financial value requested and paid/unpaid counts.

```
// server/controllers/analyticsController.js
const summary = await Booking.aggregate([
    {
        $group: {
            _id: '$status',
            count: { $sum: 1 },
            paidCount: {
                $sum: { $cond: [{ $eq: ['$paymentStatus', 'paid'] }, 1, 0] }
            },
            unpaidCount: {
                $sum: { $cond: [{ $eq: ['$paymentStatus', 'not_paid'] }, 1, 0] }
            },
            totalAmountRequested: { $sum: '$amount' }
        }
    },
    {
        $project: {
            _id: 0,
            status: '$_id',
            count: 1,
            paidCount: 1,
            unpaidCount: 1,
            totalAmountRequested: 1
        }
    },
    { $sort: { count: -1 } }
]);
```

12.2 Pipeline 2: Event Revenue & Occupancy Summary (`/api/analytics/event-summary`)

Business Goal: Cross-reference events with booking records via a server-side `$lookup` join to calculate confirmed paid revenue and occupancy percentage without counting pending or unpaid requests.

```
// server/controllers/analyticsController.js (Excerpt)
const eventSummaries = await Event.aggregate([
    {
        $lookup: {
            from: 'bookings',
            localField: '_id',
            foreignField: 'eventId',
            as: 'bookings'
        }
    },
    {
        $project: {
            title: 1,
            category: 1,
            totalSeats: 1,
            availableSeats: 1,
            ticketPrice: 1,
            totalBookings: { $size: '$bookings' },
            confirmedBookings: {
                $size: {
                    $filter: {
                        input: '$bookings',
                        as: 'b',
                        cond: { $eq: ['$$b.status', 'confirmed'] }
                    }
                }
            },
            // STRICT VERIFIED REVENUE CALCULATION:
            // Only bookings with status == 'confirmed' AND paymentStatus == 'paid'
            verifiedRevenue: {
                $reduce: {
                    input: '$bookings',
                    initialValue: 0,
                    in: {
                        $add: [
                            '$$value',
                            {
                                $cond: [
                                    {
                                        $and: [
                                            { $eq: ['$$this.status', 'confirmed'] },
                                            { $eq: ['$$this.paymentStatus', 'paid'] }
                                        ]
                                    },
                                    '$$this.amount',
                                    0
                                ]
                            }
                        ]
                    }
                }
            },
            occupancyRate: {
                $round: [
                    {
                        $multiply: [
                            { $divide: [{ $subtract: ['$totalSeats', '$availableSeats'] }, '$totalSeats'] },
                            100
                        ]
                    },
                    1
                ]
            }
        }
    },
    { $sort: { verifiedRevenue: -1 } }
]);
```

12.3 MongoDB Indexing Architecture & Query Plans

EventPulse defines targeted B-tree indexes to optimize filtering and enforce business constraints:

Collection	Index Key Pattern	Type	Query Plan Justification
`events`	`{ category: 1, date: 1 }`	Compound B-Tree	Optimizes `GET /api/events?category=Technology`. Replaces full collection scan (`COLLSCAN`) with index range scan (`IXSCAN`).
`bookings`	`{ userId: 1, createdAt: -1 }`	Compound B-Tree	Accelerates user booking history queries in reverse chronological order.
`bookings`	`{ eventId: 1, status: 1 }`	Compound B-Tree	Accelerates event-level booking lookups and status filtering.
`bookings`	`{ userId: 1, eventId: 1 }` (confirmed)	Partial Unique	Enforces uniqueness ONLY where `status == 'confirmed'`. Prevents duplicate tickets while allowing re-booking if cancelled.

Verification via explain('executionStats'): Running `node server/scripts/verify-indexes-and-queries.js` outputs execution statistics showing `totalDocsExamined: 0` and `stage: "IXSCAN"`, proving that queries read directly from index memory rather than scanning documents.

Chapter 13: Environment Setup, Seeding & Execution

13.1 Local Development Prerequisites

- **Node.js:** v18.x or v20.x
- **npm:** v9.x or higher
- **MongoDB:** v6.0+ running locally on `localhost:27017` or accessible via cloud URI.

13.2 Environment Variable Configuration (.env Template)

Create a `.env` file in the `server/` directory. *Never commit real passwords or production keys to version control!*

```
# server/.env (ILLUSTRATIVE SAFE TEMPLATE)
PORT=5000
MONGO_URI=mongodb://localhost:27017/eventpulse
JWT_SECRET=my_development_jwt_secret_key_12345
NODE_ENV=development
```

13.3 Step-by-Step Startup Commands

1. **Clone and install dependencies:**

   ```
   git clone https://github.com/jayraj0975/eventpulse-mern.git
   cd eventpulse-mern
   npm run setup   # Installs dependencies in both server/ and client/
   ```

2. **Seed the database with sample events and users:**

   ```
   cd server
   npm run seed    # Synchronizes schema indexes and seeds dummy data
   ```

3. **Start backend and frontend concurrently:**

   ```
   npm run dev     # Starts Express on :5000 and Vite on :5173
   ```

Chapter 14: Verification, Automated Testing & Known Boundaries

14.1 Automated Test Suite Execution

EventPulse contains an automated test suite executed via `npm test` in the `server/` directory:

```
$ npm test
Running AD Automated Test Suite:
✓ Test 1: Healthcheck and Events listing passed (HTTP 200)
✓ Test 2: Admin Login and JWT token issuance passed
✓ Test 3: Aggregation Pipeline 1 (Booking Status) passed with valid shape
✓ Test 4: Aggregation Pipeline 2 (Event Summary & Revenue) passed
✓ Test 5: Aggregation Pipeline 3 (Event Popularity) passed
✓ Test 6: Concurrency and duplicate active booking index constraints verified
All AD Automated Test Assertions Passed (100% Success).
```

14.2 Known Technical Boundaries & Honest Viva Disclosures

- **Multi-Document ACID Transactions:** MongoDB multi-document transactions require a MongoDB Replica Set. Because basic collegiate local development typically runs a standalone `mongod` instance, EventPulse utilizes atomic single-document operations (`findOneAndUpdate`) rather than multi-document transaction sessions.
- **Payment Gateway Simulation:** To guarantee that a student can complete a live presentation without internet or payment gateway sandbox failures, payments are simulated as internal deterministic state transitions.

Chapter 15: Step-by-Step 5-Minute Live Viva Demo Guide

> **Follow this exact 5-minute script during your practical demonstration:**

1. **Step 1: Open Home Page (30 Seconds)**
   - Open the application in your browser.
   - Point out the event cards and category filter pills (Technology, Music, Business, Art).
   - Show that each card displays **Available Seats** in real time.
2. **Step 2: Reserve a Ticket as Normal User (1 Minute)**
   - Login as `user@eventpulse.com` / `password123`.
   - Click on an event (e.g. "React & Node.js Developer Retreat").
   - Click **"Book Ticket"**. Note the immediate seat decrement.
   - Navigate to **"My Bookings"** to show the confirmed reservation.
3. **Step 3: Demonstrate Concurrency & Duplicate Prevention (1.5 Minutes)**
   - Try clicking "Book Ticket" for the same event a second time.
   - Show the alert: *"You already have an active booking for this event."*
   - **Viva Line to Say:** *"This constraint is enforced at the database layer using a MongoDB Partial Unique Index on (userId, eventId) with filter `status: 'confirmed'`."*
4. **Step 4: Login as Admin & Present Aggregation Reports (2 Minutes)**
   - Logout and log in as `admin@eventpulse.com` / `password123`.
   - Navigate to the **Admin Dashboard**.
   - Point out the 4 Overview KPI Cards (Total Verified Revenue, Occupancy Rate, etc.).
   - Show the **Booking Status Summary Table** and **Event Popularity Ranking Table**.
   - **Viva Line to Say:** *"Notice that these numbers are not computed by downloading records into JavaScript memory. They are executed directly inside MongoDB using multi-stage aggregation pipelines combining `$group`, `$lookup`, `$reduce`, and `$facet`."*

Chapter 16: Comprehensive Viva Preparation (35 Q&As)

Topic 1: MongoDB & Database Architecture

Q1: Why did you choose MongoDB over a relational database like MySQL for this project?
Answer: MongoDB stores data as flexible, self-describing BSON documents. For an event booking platform with variable event categories, embedded metadata, and rapid read requirements, MongoDB's document model allows events to be retrieved in a single read without multi-table joins. Furthermore, MongoDB's aggregation framework allows complex analytical metrics to be calculated directly in the storage engine.

Q2: What is Mongoose and what is its role?
Answer: Mongoose is an Object Data Modelling (ODM) library for MongoDB and Node.js. It provides schema validation, type casting, middleware hooks, index synchronization, and model abstraction over raw MongoDB collections (defined in `server/models/`).

Q3: How do you handle relationships in MongoDB?
Answer: EventPulse uses a hybrid referencing approach. In `Booking.js`, we store `userId` and `eventId` as `mongoose.Schema.Types.ObjectId` with `ref` declarations. For analytics, we join collections using the `$lookup` aggregation stage.

Topic 2: Concurrency & Data Integrity

Q4: How do you prevent overbooking when two users book the last seat simultaneously?
Answer: In `bookingController.js`, we execute `Event.findOneAndUpdate({ _id: eventId, availableSeats: { $gt: 0 } }, { $inc: { availableSeats: -1 } })`. Because MongoDB executes document updates atomically under the WiredTiger storage engine, the predicate check and seat decrement occur indivisibly, preventing race conditions.

Q5: What is a Partial Unique Index and why did you use it?
Answer: A standard unique index on `(userId, eventId)` would permanently prevent a user from booking an event ever again even if they cancelled their previous ticket. A Partial Unique Index adds a `partialFilterExpression: { status: 'confirmed' }`, meaning uniqueness is only enforced while the ticket is active.

Q6: How is seat restoration handled upon cancellation?
Answer: In `cancelBooking`, the server checks if the previous status was `'confirmed'` before executing `$inc: { availableSeats: 1 }`. If the booking was already pending or cancelled, seats are not restored, preventing seat inflation.

Topic 3: Aggregation Pipelines

Q7: What aggregation stages did you use in Pipeline 1?
Answer: In `/api/analytics/booking-status`, we used `$group` to bucket documents by status, `$cond` inside `$sum` to calculate paid vs unpaid counts, `$project` to reshape fields, and `$sort` to order by count descending.

Q8: How does Pipeline 2 calculate verified revenue?
Answer: In `/api/analytics/event-summary`, we join bookings via `$lookup`, and use `$reduce` with a conditional `$and` check ensuring that only bookings where `status == 'confirmed'` AND `paymentStatus == 'paid'` contribute to the revenue sum. Pending or unpaid bookings are excluded.

Q9: What is the $facet stage used for in your overview endpoint?
Answer: `$facet` allows multi-faceted aggregation, processing multiple pipeline branches within a single stage over the same incoming documents. This allows EventPulse to compute total revenue, confirmed bookings, and pending counts in a single network round-trip.

Topic 4: Indexing & Query Optimization

Q10: What indexes did you create and how do they help?
Answer: We created compound index `{ category: 1, date: 1 }` on `events` to speed up filtering by category and date; compound index `{ userId: 1, createdAt: -1 }` on `bookings` for fast user history lookups; and partial unique index `{ userId: 1, eventId: 1 }` for concurrency safety.

Q11: How do you prove that an index is working?
Answer: We append `.explain('executionStats')` to the Mongoose query. In the returned query execution plan, `stage: "IXSCAN"` confirms an index scan was used instead of a `COLLSCAN` (collection scan).

Topic 5: Questions the Examiner May Ask About Implementation Quirks

Q12: Why didn't you use multi-document ACID transactions with session.startTransaction()?
Answer: MongoDB multi-document transactions require a multi-node Replica Set. In local collegiate setups and standalone Docker deployments, replica sets are often not configured. We achieved concurrency safety without replica set overhead by using atomic single-document updates with conditional predicates.

Q13: Why did you rebrand from Eventora to EventPulse?
Answer: To clearly delineate our enhanced, database-focused academic contribution from the upstream repository, while ensuring complete isolation from unrelated personal or production databases.

Q14: Where does the JWT token live in the client?

Answer: In `localStorage`. It is extracted by an Axios request interceptor and injected into the `Authorization: Bearer` header for every authenticated request.

Q15: What happens if MongoDB goes down while the server is running?

Answer: Mongoose's connection buffer holds requests temporarily while attempting to reconnect. If reconnection fails within the timeout window, Mongoose throws a `MongooseServerSelectionError`, which our Express route error handlers catch to return HTTP 500 responses.

Chapter 17: Glossary, Quick Reference & Summary Sheets

17.1 Important Technical Terms

- **BSON:** Binary JSON. The binary-encoded serialization format used by MongoDB to store documents and perform remote procedure calls.
- **IXSCAN:** Index Scan. An execution plan state proving that MongoDB retrieved records using an in-memory B-tree index rather than scanning every document on disk.
- **COLLSCAN:** Collection Scan. An inefficient execution plan state where MongoDB scans every document in a collection because no suitable index was found.
- **Partial Unique Index:** An index that only enforces uniqueness on documents that satisfy a specified filter expression (e.g. `status: 'confirmed'`).
- **Atomic Operation:** An operation that completes entirely or not at all, with no possibility of another concurrent process seeing an intermediate state.
- **$lookup:** A MongoDB aggregation stage performing an equality left outer join between two collections in the same database.

17.2 One-Page Quick Revision Sheet

Topic	Key Fact to State in Viva	Code Reference
Seat Allocation	Atomic `findOneAndUpdate` with `{ availableSeats: { $gt: 0 } }` and `{ $inc: { availableSeats: -1 } }`	`server/controllers/bookingController.js`
Active Duplicate Guard	MongoDB Partial Unique Index on `(userId, eventId)` with filter `status: 'confirmed'`	`server/models/Booking.js`
Seat Restoration Guard	Restores seat on cancellation ONLY if previous status was confirmed.	`cancelBooking` in `bookingController.js`
Pipeline 1 (Status)	`$group` by status, conditional `$cond` paid/unpaid accumulation, `$sort`	`/api/analytics/booking-status`
Pipeline 2 (Revenue)	`$lookup` join, strict `$reduce` verified revenue (paid + confirmed only), occupancy rate	`/api/analytics/event-summary`
Pipeline 3 (Popularity)	`$match: { status: 'confirmed' }`, `$group` by eventId, `$lookup` title/price, `$sort`	`/api/analytics/event-popularity`
Overview KPIs	Single-pass `$facet` aggregation computing 7 KPIs in one network trip.	`/api/analytics/overview`